PTCS-RL:Progressive Tool Call Scheduling Reinforcement Learning for Mathematical Reasoning

Jukun Liang[a], Mingtao Zhou[a], Jianhou Gan[a,b,*], Juxiang Zhou[a,b] and Huibang Shi[a]

[a]Key Laboratory of Educational Informatization for Nationalities, Ministry of Education, Yunnan Normal University, Juxian Street 768, Kunming, 650500, China

[b]Yunnan Key Laboratory of Smart Education, Yunnan Normal University, Juxian Street 768, Kunming, 650500, China

HIGHLIGHTS

- PTCS-RL for mathematical reasoning with progressive tool-call scheduling.
- Execution feedback improves verification and error correction in RL training.
- PTCS-RL outperforms representative baselines, especially on challenging benchmarks.

ARTICLE INFO

Keywords:
Large language models
Mathematical reasoning
Tool-augmented reasoning

ABSTRACT

In recent years, large language models have made significant progress in mathematical reasoning. However, when faced with complex mathematical problems and multi-step reasoning tasks, they are prone to error accumulation, which undermines reasoning accuracy and solution reliability. To address this issue, this paper proposes Progressive Tool Call Scheduling Reinforcement Learning (PTCS-RL). The proposed method introduces an external code execution tool into reinforcement learning training, delegates critical intermediate computations to the tool, and feeds the execution results back into subsequent reasoning. This enables the model to verify and revise its reasoning based on tool feedback, thereby mitigating error accumulation. In addition, to reduce interaction complexity and noise during training, the method imposes a low tool-call budget in the early stages of training and gradually relaxes this budget in later stages, guiding the model to learn more sophisticated multi-step tool-use and feedback-driven interaction strategies. Experimental results show that the proposed method achieves superior overall performance across multiple datasets, outperforming the strong baseline ToRL-1.5B by 4.78%, with particularly notable gains on challenging benchmarks such as AIME.

1. Introduction

Large language models (LLMs) have widely adopted Chain-of-Thought (CoT) [1] reasoning to solve complex mathematical problems. However, this approach is highly prone to calculation hallucinations and error accumulation, which directly undermine the accuracy and reliability of solutions. To fundamentally address these computational limitations, Tool-Integrated Reasoning (TIR) [2] has emerged. TIR equips models with the ability to generate code, allowing them to delegate complex computations to an external Python interpreter [3, 4, 5]. However, most existing TIR methods rely on supervised fine-tuning (SFT) [6, 7]. SFT-based training confines models to manually predefined static trajectories, which not only hinders autonomous exploration of optimal tool-use strategies but also limits feedback-based self-correction when the model encounters unseen problems or tool execution errors.

As reinforcement learning (RL) has shown strong effectiveness in improving the reasoning capabilities of large models [8, 9, 10], recent studies have begun to apply RL directly to tool-augmented reasoning [11, 12, 13]. However, this introduces a serious optimization challenge: tool execution significantly lengthens interaction trajectories and introduces additional noise, making policy collapse more likely in the early stages of training. To ensure stability, existing methods often adopt a compromise strategy by imposing a fixed tool-call budget to truncate long interactions [11, 14]. Although a fixed budget helps control training noise in the early stages, it restricts the model's exploration space once its capabilities improve, making it difficult to perform multi-step verification and complex error correction, and thus limiting reasoning accuracy on challenging mathematical problems.

To overcome the exploration bottleneck caused by static budgets, this paper proposes a progressive tool-call scheduling reinforcement learning method, PTCS-RL, based on Qwen2.5-Math-1.5B [15]. As illustrated in Fig 1, the proposed method differs substantially from pure-text CoT reasoning. When handling complex large-number calculations, pure-text CoT is prone to generating and propagating computational errors. In contrast, our reinforcement-based mechanism allows the model to decide when to halt generation, wait for sandbox execution, and continue reasoning only

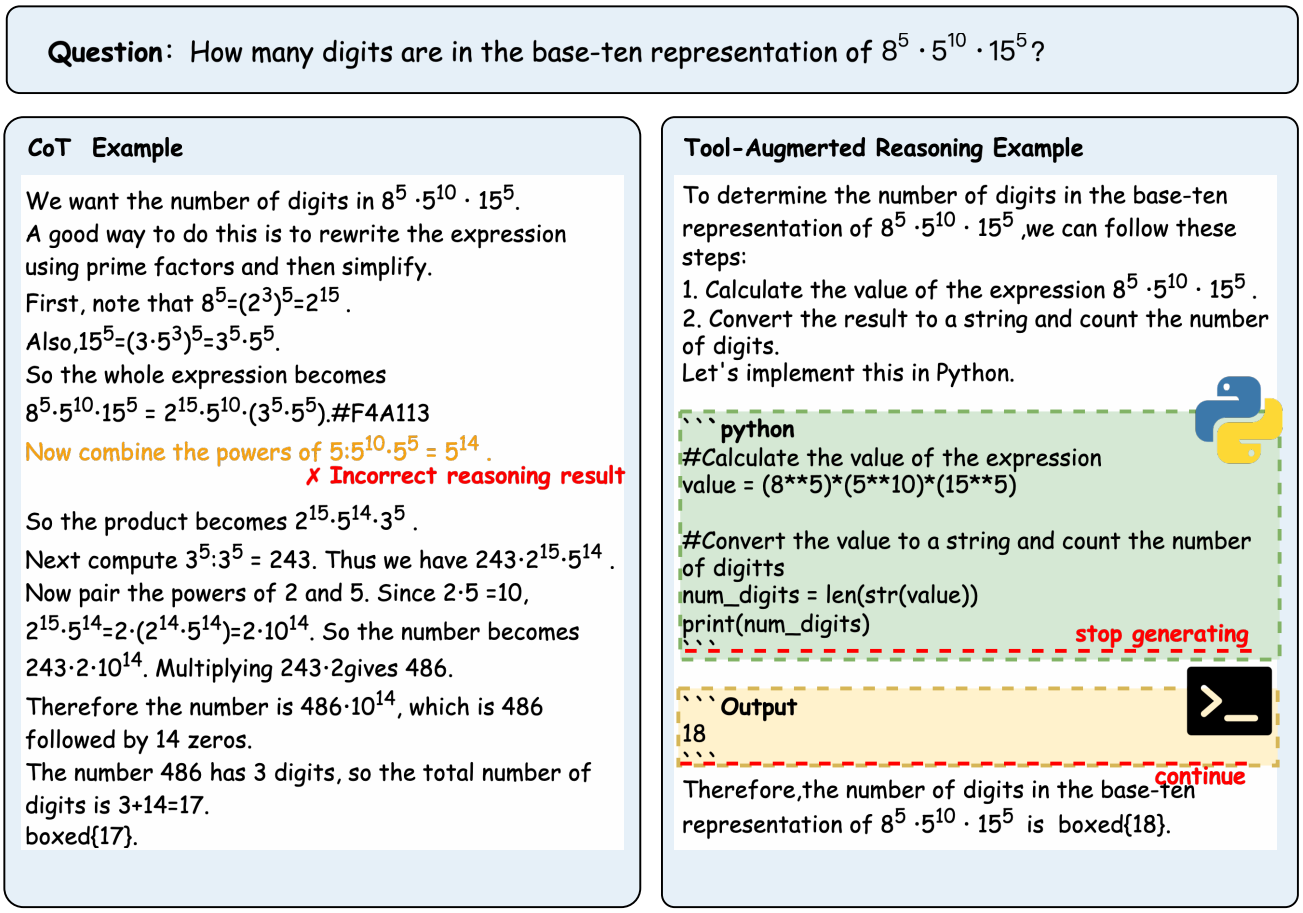


Fig. 1. Comparison between pure-text CoT reasoning and tool-augmented reasoning.

after receiving precise numerical results, thereby effectively mitigating error propagation. In addition, the progressive scheduling mechanism dynamically adjusts the interaction budget across training stages: strict limits are imposed in the early stages to reduce noise, and the budget is gradually relaxed as training progresses, guiding the model to develop more sophisticated multi-step tool-use strategies.

The contributions of this paper are as follows:

1. We propose PTCS-RL for mathematical reasoning, which dynamically adjusts the tool-call budget to alleviate training instability and restricted exploration caused by fixed-budget strategies.
2. We incorporate an external execution tool into RL training, enabling the model to verify and revise its reasoning based on tool feedback, thereby improving accuracy and reliability on complex mathematical tasks.
3. Experiments on multiple mathematical reasoning benchmarks show that the proposed method outperforms representative baselines overall and achieves more pronounced gains on challenging tasks.

2. Related Work

2.1. Mathematical Reasoning with Large Language Models

LLMs have achieved substantial progress in mathematical reasoning. Early advances were driven by Chain-of-Thought prompting [1], which improves logical clarity by guiding models to generate intermediate reasoning steps. To further enhance the fault tolerance and robustness of pure text-based reasoning, methods such as self-consistency [16], Tree of Thoughts [17], and ReAct [18] have systematically expanded the reasoning boundaries of models in complex problem-solving spaces. Specifically, these methods rely on majority voting over independently sampled reasoning paths, heuristic multi-branch tree search, and the interleaving of internal reasoning with external state updates, respectively. In recent years, large models have shown rapid progress in large-scale post-training with reinforcement learning. As demonstrated by the strong performance of DeepSeek-R1 [8] and OpenAI o1 [9], models can develop ultra-long-chain self-reflection, backtracking, and error-correction capabilities using only reinforcement learning objectives and scalar outcome rewards, without relying heavily on high-quality human-annotated step-level

supervision. In this context, to address the extreme sparsity of final-result rewards, researchers have also developed process reward models. For example, Math-Shepherd [19] can automatically assign fine-grained step-level verification scores through Monte Carlo tree search without human-annotated step-level labels. Similarly, methods such as Eurus [20] explore implicit reward modeling to provide dense local supervision for reinforcement learning optimization. However, due to the probabilistic nature of autoregressive generation, pure language models still lack rigorous verification in complex mathematical derivations, making it difficult to overcome the bottleneck of calculation hallucinations.

2.2. Supervised Fine-Tuning for Tool-Augmented Reasoning

To overcome the computational bottlenecks discussed above, incorporating external tools into the reasoning process has become a mainstream approach. Early pioneering works such as PAL [3] and PoT [4] proposed that models focus on logical reasoning while delegating error-prone arithmetic computations to an external Python interpreter. Toolformer [6] further demonstrated the potential of alternating between text generation and tool use through self-supervised placeholder-based API invocation. With the rapid evolution of hybrid reasoning methods, ToRA [5] and MathCoder [21] developed integrated frameworks that seamlessly connect natural language reasoning with program execution, substantially improving performance on multi-step mathematical problem solving. MATHSENSEI [22] further extends this paradigm by integrating a variety of specialized tools, including web search, Python sandboxes, and even the WolframAlpha API. In addition, interactive prompting methods such as IMR-TIP [23] introduce multi-perspective tool feedback to enable cross-validation of intermediate reasoning results. The START [24] framework also exploits extremely long contexts to tightly integrate ultra-long-chain reasoning with real-time code execution. However, most existing tool-augmented models still rely on supervised fine-tuning. supervised fine-tuning is essentially a form of behavior cloning, whose objective is to imitate predefined expert trajectories rather than directly maximize final-answer correctness. As a result, the model is confined to static interaction patterns. When faced with out-of-distribution problems or previously unseen code execution errors, it often fails to perform effective self-correction because it lacks sufficient experience in exploration and trial-and-error, leading to limited generalization ability and fault tolerance.

2.3. Reinforcement Learning Challenges and Tool Budget Scheduling

To overcome the generalization limitations imposed by SFT data distributions, recent research has rapidly shifted toward tool-augmented reinforcement learning. Instead of relying on manually constructed expert trajectories, these methods use final-answer correctness as the reward signal and employ reinforcement learning algorithms, such as PPO and its variants, to drive autonomous exploration in interactive environments. For example, ToRL [11] incorporates interpreter environment states into the reinforcement learning update loop, enabling the model to decide when to invoke tools. Subsequent studies, including ReTool [12], Agent-Math [13], ARTIST [25], and TAPO [26], further show that models trained with code execution feedback and trial-and-error rewards exhibit greater flexibility than SFT-based approaches in multi-round tool scheduling and error correction. However, combining reinforcement learning with external tools also introduces substantial training instability. Code compilation failures, execution timeouts, and highly variable feedback can all increase exploration variance. In the early stages of training in particular, consecutive failures often lead to sparse rewards, which may trigger policy oscillation or even training collapse. To ensure convergence, existing methods such as ToRL [11] and CIR [14] typically adopt precise boundary matching and fixed tool-call budget strategies. Specifically, they impose a predefined upper bound on tool usage and suppress exploration variance by truncating long interaction sequences. Although this design helps improve training stability in the early stages, it also constrains the model's exploration of multi-step tool collaboration strategies in later stages, making it difficult to learn more sophisticated long-chain verification and error-correction behaviors. To address this limitation, this paper proposes a progressive tool-call scheduling method that imposes strict constraints in the early stage and gradually relaxes them as training proceeds, thereby maintaining training stability while promoting the acquisition of multi-step collaborative strategies.

3. Methodology

To provide a more intuitive illustration of the progressive tool-call scheduling reinforcement learning framework proposed in this paper, Fig. 2 presents an overview of the overall methodology. This framework integrates the natural language reasoning capabilities of LLMs with external tool execution processes and employs reinforcement learning to optimize both reasoning and tool-use strategies. In addition, a dynamic tool-call scheduling mechanism is introduced to gradually adjust the tool-call budget during training, thereby balancing training stability with the ability to perform multi-step tool interactions.

3.1. Formulation of the Tool-Augmented Reinforcement Learning Problem

This paper formally defines the tool-augmented reinforcement learning problem. Specifically, we construct a reasoning-based learning framework that incorporates external tools into the reinforcement learning loop: when generating reasoning trajectories, the model can produce tool-invocation actions and use the returned execution results as objective evidence for subsequent decisions, thereby supporting verifiable computation and step-by-step problem solving in complex tasks. Furthermore, during training, this paper introduces a progressive tool-call scheduling method

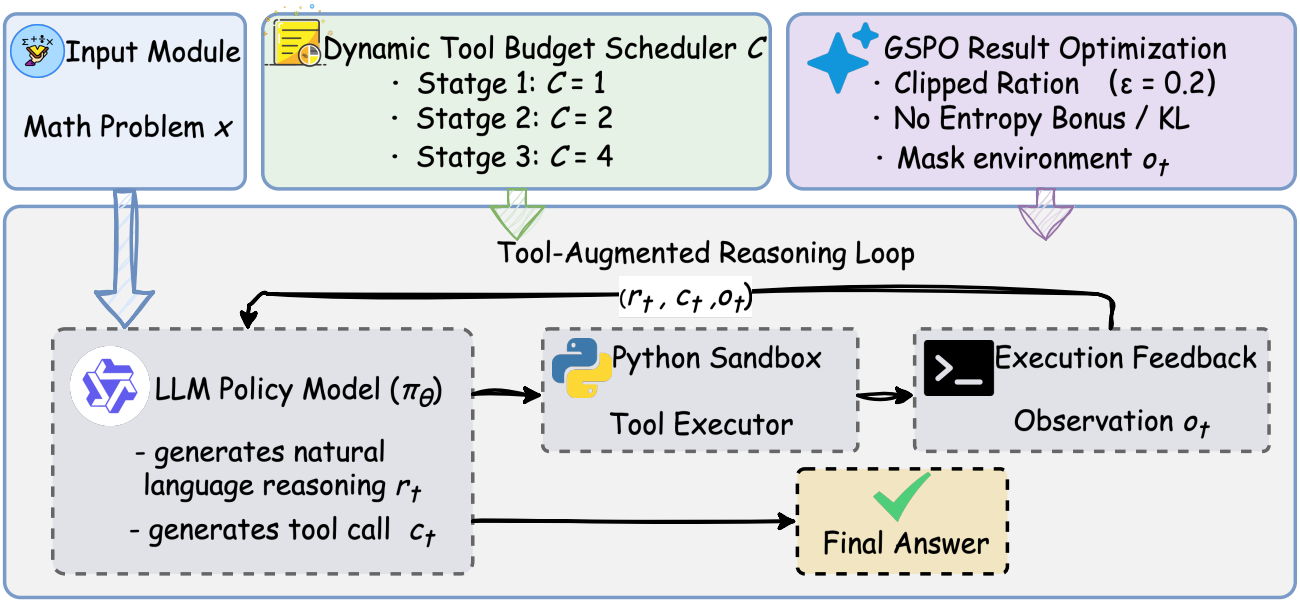


Fig. 2. Progressive tool-call scheduling tool-augmented reinforcement learning framework.

that adjusts the tool-call budget in stages as training proceeds, while a fixed tool-call budget is adopted during inference. Concretely, given a problem x, the model is required to generate a complete reasoning trajectory that integrates both natural language reasoning and tool interaction. Unlike traditional reinforcement learning based solely on natural language reasoning, tool-augmented reinforcement learning allows the model to explicitly invoke external tools during the reasoning process and condition subsequent reasoning on the execution results returned by these tools. Under this setting, the trajectory generated by the model can be represented as:

$$\tau = (r_1, c_1, o_1), (r_2, c_2, o_2), \ldots, (r_t, c_t, o_t) \tag{1}$$

Here, r_t denotes the natural language reasoning content at step t, c_t denotes the tool invocation generated by the model at that step, and o_t is the result returned after the corresponding tool is executed. During the reasoning process, the model generates the trajectory above in an autoregressive manner, where each step is conditioned on the original input x and all previously generated reasoning content. The execution results returned by the tools are explicitly incorporated into the context, thereby influencing subsequent reasoning and decision-making. The objective of reinforcement learning is to optimize the model parameters so that they maximize the outcome-based reward signal in a tool-augmented reasoning environment. Unlike process-based supervision, this paper adopts an outcome-oriented reward design, in which rewards are assigned solely according to whether the model's final answer is correct, without imposing explicit supervision on intermediate reasoning steps or tool invocations. Formally, the model is trained by maximizing the following expected objective:

$$\max_{\theta} \ \mathbb{E}_{\tau \sim \pi_\theta(\cdot|x)} \left[R(x, \tau) \right] \tag{2}$$

Here, π_θ denotes the current policy model, and $R(x, \tau)$ denotes a scalar reward function that maps the input x and the generated trajectory τ to a single scalar value used to evaluate the overall quality of the trajectory.

It should be noted that the introduction of tool calls causes inference trajectories to include both model-generated actions and execution results returned by the external environment, thereby introducing a certain degree of noise into the trajectories. This characteristic poses new challenges for tool-augmented reinforcement learning in terms of policy optimization and training stability, beyond those encountered in traditional reinforcement learning. Subsequent sections will further investigate policy optimization methods and training mechanisms suitable for tool-augmented reasoning scenarios.

3.2. GSPO-Based Tool-Augmented Reinforcement Learning

Under the tool-augmented reinforcement learning framework defined in the previous subsection, the model must optimize its policy during training based on hybrid trajectories that integrate both natural language reasoning and tool interaction. Because these reasoning trajectories contain both model-generated textual content and explicit tool-invocation actions, the action space is decomposed during policy modeling to distinguish between the two controllable behavior types: text generation and tool invocation.

More specifically, at time step t, the model's action can be represented as a tuple:

$$a_t = (r_t, c_t) \tag{3}$$

Here, r_t denotes the natural language reasoning content generated by the model, and c_t denotes the tool-related reasoning content. Given the state s_t, the policy model can factorize the probability of the action as:

$$\pi_\theta(a_t \mid s_t) = \pi_\theta(r_t \mid s_t)\, \pi_\theta(c_t \mid s_t) \tag{4}$$

This factorization characterizes the "dual-action" decision process under a unified policy: given the state s_t, π_θ jointly models both text generation r_t and tool invocation c_t. The execution result of the tool call is returned by the external tool as observational feedback and incorporated into the subsequent state, thereby influencing the following reasoning process.

Under this action modeling scheme, policy optimization applies only to the model's text generation and tool-invocation behavior. Given an input problem x, the model samples a tool-augmented reasoning trajectory τ under the old policy $\pi_{\theta_{\text{old}}}$. In this paper, we adopt GSPO (Group Sequence Policy Optimization) [27], whose algorithmic core is derived from the key variant of group-relative policy optimization, GRPO [28], to update the policy model. The optimization objective over tool-augmented trajectories is defined as follows:

$$\mathcal{L}_{\text{GSPO}}(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta_{\text{old}}}} \left[\sum_t w_t \cdot \frac{\pi_\theta(r_t, c_t \mid s_t)}{\pi_{\theta_{\text{old}}}(r_t, c_t \mid s_t)} \cdot \hat{A}_t \right] \tag{5}$$

Here, $\hat{A}_t$ denotes the advantage estimate at time step t, and w_t is a stability weighting term used to regulate the contribution of different time steps to the policy update. This objective jointly models text generation and tool invocation, while ensuring that the policy gradient is produced only by the model's own controllable decisions.

Compared with common policy optimization methods based on clipped objectives, GSPO adopts a smoother policy update mechanism to regulate the magnitude of changes in the policy distribution, making it more suitable for complex generation trajectories that involve both textual reasoning and tool invocation. Policy optimization based on GSPO provides a stable training foundation for learning effective reasoning and tool-use strategies in tool-augmented reasoning environments, and also establishes the optimization basis for the subsequent introduction of dynamic tool-call scheduling.

However, effective tool-augmented reinforcement learning also requires proper scheduling of tool use during training.

3.3. Progressive Training and Tool-Call Scheduling Strategy

In tool-augmented reinforcement learning scenarios, models must be trained in environments involving long-sequence reasoning and multi-stage tool interactions, which impose higher demands on both training stability and training efficiency. As observed in Experiment 5.2.2, introducing high-frequency tool calls and multi-round interactions too early in multi-step reinforcement learning training makes the optimization process much more prone to instability, and in extreme cases may even lead to training collapse. Moreover, because tool invocation typically incurs nontrivial interaction cost, introducing multiple tool interactions too early or too aggressively can substantially increase the time overhead during training. Therefore, a central challenge in tool-augmented reinforcement learning is how to progressively improve the model's tool-use capability while maintaining stable training.

To address these challenges, existing studies generally constrain tool interaction behavior during training in order to reduce training noise and improve stability. Among them, ToRL [11] imposes a fixed upper bound on the tool-call budget during training to control the interaction complexity of a single trajectory, while CIR [14] further emphasizes improving the trainability of tool-augmented reinforcement learning under multi-step interaction settings through dedicated training strategies and stabilization mechanisms. Taken together, these studies suggest that appropriately constraining the tool interaction process is an effective way to improve both the stability and the efficiency of tool-augmented reinforcement learning.

Within this general improvement framework, and as illustrated in Fig. 2, this paper presents an integrated training design for tool-augmented reinforcement learning from three perspectives: policy optimization, interaction boundary control, and tool-call scheduling mechanisms.

Policy optimization: Based on GSPO, this paper applies a fixed clipping constraint of 0.2 to the policy ratio in order to limit the magnitude of changes in the policy distribution during each update and improve the stability of multi-step reinforcement learning training. At the same time, entropy rewards are disabled and KL-divergence regularization is removed during training to reduce the interference of additional exploration noise and overly strong constraints on policy learning.

Interaction boundary control: This paper explicitly distinguishes between model-generated behavior (r_t, c_t) and feedback returned by external tools (o_t), so that policy optimization is applied only to controllable natural language reasoning and tool-invocation actions produced by the model, while tool execution results are treated solely as environmental observations inserted into the subsequent context and do not participate in gradient updates.

Tool-call scheduling mechanism: The tool-call budget is defined as a scheduling variable that changes dynamically with the training process. In the early stages of training, the number of tool calls is restricted to reduce interaction

complexity and feedback noise; in the later stages, the budget constraint is gradually relaxed to support more complex multi-step tool interaction and self-correction based on execution feedback. Detailed settings are provided in Section 4.3.

4. Experimental Setup

4.1. Datasets

To comprehensively evaluate the model's performance on mathematical reasoning tasks of varying difficulty, this paper selects representative datasets for both training and testing. For training, the model is optimized with reinforcement learning using the high-quality mathematical reasoning corpus constructed in the ToRL study [11]. This training set covers core problem types such as algebra, geometry, and number theory, while maintaining a well-balanced, progressive distribution of difficulty levels. It effectively guides the model to explore and construct an interactive closed loop of reasoning, code generation, and execution feedback during training. For evaluation, this paper adopts four mainstream mathematical reasoning benchmarks to build a progressive test environment ranging from basic arithmetic to the difficulty of top-level international competitions.

GSM8K [29]: This dataset consists of high-quality grade-school mathematical word problems and is mainly used to evaluate the model's basic arithmetic reasoning and single-step logical understanding.

AMC23 [30]: This benchmark contains authentic problems from the 2023 American Mathematics Competitions (AMC), whose questions generally involve moderate to high logical complexity and abstract modeling requirements.

AIME24 and AIME25 [30]: These benchmarks are drawn from recent high-difficulty problems in the American Invitational Mathematics Examination (AIME). Such problems require multi-step reasoning chains and extremely high computational precision, where even minor arithmetic deviations can lead to substantial cumulative errors. They therefore serve as core evaluation benchmarks for measuring the model's upper-bound capability in complex reasoning and multi-step tool-coordination tasks.

4.2. Comparison Models

To objectively verify the effectiveness of the progressive tool-call scheduling mechanism proposed in this paper, we select comparison baselines that cover base models, instruction-tuned models, and large-parameter reinforcement learning models.

Base and instruction-tuned models: Qwen2.5-Math-1.5B [15] is used as the base model in our experiments. This model has not undergone instruction tuning or additional reinforcement learning and is therefore used to measure the fundamental mathematical reasoning capability of a model at this scale. Qwen2.5-Math-1.5B-Ins is the corresponding instruction-following model obtained through supervised fine-tuning. On top of this, we additionally introduce a Qwen2.5-Math-1.5B-Ins (with Code) setting, which allows the model to generate and execute code during testing. This setting enables a direct evaluation of the specific effect of tool use during inference on reasoning accuracy.

Reinforcement learning reasoning models: SimpleRL-Zoo-7B [31] and Eurus-2-7B-PRIME [20] are two representative 7B-scale models whose reasoning capabilities are improved through conventional pure-text reinforcement learning training. Both exhibit strong performance on mainstream mathematical benchmarks. We include them as comparison targets to show the level of reasoning performance that can be achieved by larger-scale reinforcement learning models.

Tool-augmented reinforcement learning model: ToRL-1.5B [11] serves as a representative current method for tool-augmented reinforcement learning and is also the most important baseline in this paper. During training, this model learns tool-use strategies by imposing a static fixed tool-call budget. A direct comparison between this static scheduling design and the dynamic progressive scheduling mechanism proposed in this paper provides the clearest evidence of the overall accuracy gains brought by our method across mathematical benchmarks of varying difficulty.

4.3. Hyperparameter Settings

This section describes the core hyperparameters and software-hardware infrastructure used for model training, external code execution, and reasoning evaluation in order to ensure reproducibility. All experiments are conducted on a server equipped with dual NVIDIA A100 (80GB) GPUs and a 196-core CPU. Model training and inference are implemented based on the open-source reinforcement learning framework `verl`. To support external code execution, we deploy the SandboxFusion sandbox environment on the server. This environment provides isolated execution containers for model-generated code and returns standardized execution results to the model as contextual feedback for subsequent states.

Policy optimization parameters: During reinforcement learning updates, this paper adopts GSPO as the policy optimization algorithm. To ensure stability in long-trajectory training with tool usage, the clipping range of the policy ratio is strictly set to 0.2. To avoid introducing unnecessary exploration noise during the tool-scheduling learning phase, entropy rewards and KL-divergence regularization are removed, thereby granting the model greater flexibility in adjusting the policy distribution. The global training configuration is as follows: the batch size is set to 128, the number of sampled generations is 16, and the sampling temperature is fixed at 1.0.

Tool-call scheduling configuration: Under the progressive tool-call scheduling strategy proposed in this paper, the training process is divided into three stages, namely Early, Middle, and Late. For each stage, a corresponding training-step interval and upper bound on tool calls are specified. The detailed configuration is shown in Table 1.

Inference evaluation settings: During the generation phase of benchmark evaluation, all comparison models use greedy decoding with temperature $T = 0$ for logical reasoning generation, in order to eliminate the effect of random sampling.

Final answers are extracted from the model outputs using an answer-parsing module.

Table 1
Progressive tool-call budget scheduling strategy.

Stage	Training Steps	Static-C
Early	0–400	1
Middle	401–500	2
Late	501–600	4

5. Results and Analysis

5.1. Results

We conduct a comprehensive evaluation of different types of models on four mathematical reasoning benchmarks, and the detailed quantitative comparison is reported in Table 2. Overall, as the training paradigm evolves from base pretraining to instruction tuning and further to tool-augmented reinforcement learning, model performance on complex mathematical reasoning tasks shows a clear upward trend. The unoptimized base model Qwen2.5-Math-1.5B [15] performs very poorly on competition-level problems that require long reasoning chains and are highly sensitive to numerical precision, achieving accuracies of only 6.7% and 4.6% on AIME24 and AIME25, respectively. Although the instruction-tuned model Qwen2.5-Math-1.5B-Ins achieves substantial gains on medium-difficulty tasks such as GSM8K and AMC23, simply granting this model temporary access to external code execution only at inference time, namely under the with-Code setting, yields only a slight improvement on AIME to 13.3%, while its performance on AMC23 declines. This phenomenon clearly indicates that merely enabling code tools during inference is not sufficient. Without training the model to learn robust tool invocation and feedback integration strategies, the benefits of tool use on out-of-distribution computational tasks remain highly unstable.

In addition, we evaluate two 7B-scale reinforcement learning reasoning models, SimpleRL-Zoo-7B [31] and Eurus-2-7B-PRIME [20]. Although the larger parameter scale and stronger knowledge capacity enable them to surpass 91% accuracy on the relatively basic GSM8K benchmark, such pure text-generation models still exhibit a clear performance collapse on the highly challenging AIME25 benchmark that requires extreme verification accuracy. For example, Eurus achieves only 6.7% on AIME25. This further confirms that in mathematical exploration tasks characterized by large reasoning depth and high numerical precision, simply scaling model parameters cannot replace the deterministic execution feedback provided by external computational tools.

After introducing the tool-augmented reinforcement learning paradigm, the representative mainstream baseline ToRL-1.5B [11], which uses a static fixed-budget strategy, further improves the performance of a small-scale base model on complex mathematical reasoning tasks and reaches an average accuracy of 51.55%. In comparison, the progressive tool-call scheduling method proposed in this paper establishes performance gains on all four benchmarks and further raises the overall average accuracy to 56.33%, yielding an absolute improvement of 4.78% over the core baseline. More importantly, these gains are concentrated on the most difficult benchmarks, especially AIME24, where the improvement reaches 10.0%, and AIME25. This result strongly demonstrates that by progressively relaxing the tool-call budget, the model is able to overcome the observational noise and training oscillation accumulated during the early long-horizon exploration phase, and ultimately acquire multi-step self-verification capability based on execution feedback in the middle and late stages of training. As a result, it breaks through the accuracy ceiling of pure text-based logical reasoning while still using a relatively small base model.

5.2. Analysis of the Progressive Tool-Call Scheduling Method

5.2.1. Analysis of Tool-Use Behavior

To analyze how the model gradually develops tool-use behavior during tool-augmented reinforcement learning training, this paper conducts a statistical analysis of key training-stage indicators. Specifically, we record the average number of intended tool calls, the ratio of code generation, and the average response length during training, in order to examine the dynamic evolution of tool-use behavior under different tool-budget strategies.

We first analyze the model's tool-use intention under a fixed tool budget. As shown in Fig. 3, when the tool-call budget is fixed at 1 during training, namely Static-C=1, the average number of intended tool calls rises rapidly in the early stage of training and stabilizes at around 1.4. In the later stage of training, this value further increases to approximately 1.6–1.7. It is important to note that the dashed line in the figure represents the maximum tool-call budget allowed during training, namely Tool-Budget=1. It can be seen that the average number of intended tool calls remains consistently above this budget limit throughout training. This indicates that although the training environment permits only one actual tool call, the model still tends at the policy level to initiate multiple rounds of tool interaction, thereby exhibiting a clear budget-exceeding tendency in its tool-use intention. In other words, a fixed low tool budget imposes a hard constraint on behavioral formation during training, while the model gradually reveals an intrinsic need for multi-step tool interaction in its reasoning strategy.

After establishing this behavior-mismatch phenomenon, we further analyze the model's code-generation behavior under different tool-budget strategies. As shown in Fig. 4(a), all three strategies exhibit a rapid increase in the code-generation ratio during the early stage of training, indicating that the model quickly learns to transform part of the reasoning process into executable code under reinforcement learning. However, clear differences begin to emerge in the middle and later stages of training. Under Static-C=1, the code-generation ratio grows more slowly overall, suggesting that the model's ability to solve problems through code is

Table 2
Accuracy of different models on mathematical benchmarks.

Model	Code	GSM8K	AMC23	AIME24	AIME25	Avg. acc
Base & Ins Model						
Qwen2.5-Math-1.5B	×	74.6	28.0	6.7	4.6	28.48
Qwen2.5-Math-1.5B-Ins	×	84.8	62.5	10.0	10.0	41.75
Qwen2.5-Math-1.5B-Ins	✓	83.7	55.0	13.3	13.3	41.33
RL Reasoning Model						
SimpleRL-Zoo-7B	×	91.7	62.5	20.0	16.7	47.73
Eurus-2-7B-PRIME	×	**92.0**	50.6	20.0	6.7	42.32
Tool-Augmented RL Model						
ToRL-1.5B	✓	85.3	67.5	26.7	26.7	51.55
PTCS-RL-1.5B (Ours)	✓	88.6	**70.0**	**36.7**	**30.0**	**56.33**

constrained by the single-call tool budget. Although Static-C=4 improves the code-generation ratio in the early and middle stages, it exhibits noticeable fluctuations around step 350, revealing a degree of training instability. In contrast, the Progressive Tool Budget strategy shows a smoother and more sustained upward trend throughout training and reaches a higher code-generation ratio in the later stage. This indicates that progressively relaxing the tool-call budget enables the model to gradually adapt to more complex tool-interaction behavior, while avoiding the instability caused by excessive tool-interaction freedom in the early phase of training.

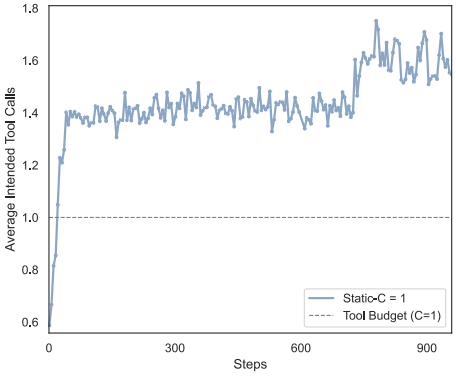
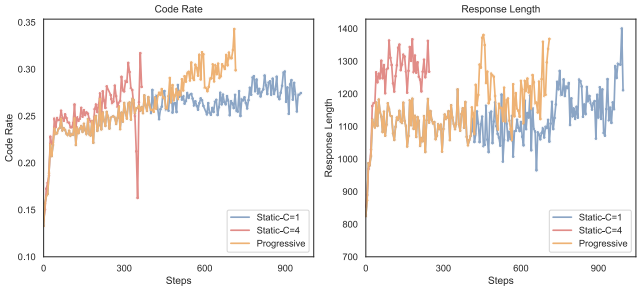


Fig. 3. Variation in the average number of intended tool calls under the fixed tool budget Static-C=1.

In addition to the code-generation ratio, this paper also examines the change in response length under different strategies. As shown in Fig. 4(b), Static-C=4 exhibits a relatively larger response length in the early stage of training, indicating that a larger tool-call budget tends to induce the model to generate longer reasoning and code trajectories. In contrast, the response length under Static-C=1 increases markedly in the later stage of training, which may suggest that when limited to a single tool call, the model compensates by increasing the number of natural language reasoning steps or embedding more computational logic into a single tool invocation. By comparison, the Progressive Tool Budget strategy maintains a relatively smooth increase in response length over most of training, and only shows a reasonable upward trend in the later stage as the tool budget is gradually relaxed. This reflects a more stable coordination between reasoning complexity and tool-interaction capability.



(a) Code-generation ratio (b) Response length

Fig. 4. Training dynamics of model reasoning behavior under different tool-budget strategies. Panel (a) shows the code-generation ratio, and panel (b) shows the response length.

Taken together, Fig. 3 and Fig. 4 show that under a fixed tool budget, the model gradually develops a clear need for multiple tool calls in the middle and later stages of training. In contrast, the progressive tool-budget strategy allows the model's code-generation behavior and reasoning complexity to evolve more smoothly as training proceeds by gradually relaxing the upper bound on tool calls. As a result, it stabilizes training while progressively improving the model's ability for tool-augmented reasoning.

5.2.2. Comparison Between Progressive Scheduling and a Fixed Tool Budget

To further analyze the role of progressive tool-call scheduling, this paper compares the training dynamics of a fixed tool budget, namely Static-C=4, and dynamic scheduling, namely the Progressive Tool Budget strategy, under the same model scale Qwen2.5-Math-1.5B [15] and the same

reinforcement learning setting on the ToRL dataset [11], as shown in Fig. 5. Under the fixed tool budget, the accuracy of Static-C=4 can rise to approximately 0.43–0.45 in the early stage of training. However, after around step 280, the fluctuations increase significantly, and a clear performance collapse appears around step 350, where the accuracy drops sharply. This indicates that when the model is exposed to a high degree of freedom in tool scheduling too early, policy updates become much more unstable. In contrast, the curve of the progressive tool-call scheduling method is overall much smoother, and the accuracy improves steadily throughout training. Therefore, this comparison shows that gradually relaxing the upper bound on tool calls can reduce interaction complexity and optimization noise in the early training stage, allowing a small-parameter model to first acquire basic reasoning and single-step tool-use capability and then gradually adapt to multi-step tool interaction, thereby achieving a more stable training process and better final performance.

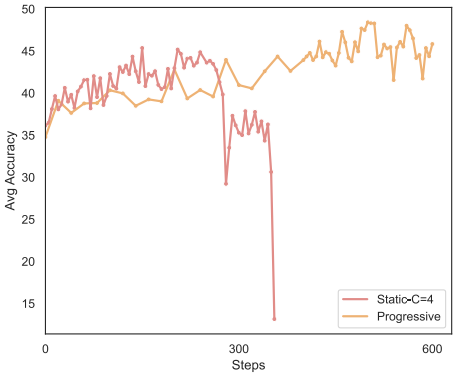


Fig. 5. Accuracy comparison between progressive scheduling and a fixed tool budget.

5.2.3. Training Cost

To analyze the impact of different tool-call budgets on training cost, this paper measures and compares the training process of three fixed-budget Static-C strategies under the same model scale Qwen2.5-Math-1.5B [15], the same training configuration, and the same data-sampling setting, and reports the average step time for each strategy, as shown in Table 3. Here, the "average step time" is measured in wall-clock time and covers the complete pipeline of a given training step, including rollout, sandbox execution, and policy updates, thereby reflecting the real system overhead introduced by tool interaction.

Table 3
Comparison of average step training time under different static tool-call budgets.

Static-C	Average Step Time (s)
1	400
2	550
4	800

As shown in Table 3, as the static tool budget increases from 1 to 2 and then to 4, the average step time rises substantially: the average step time under Static-C=1/2/4 is 400s/550s/800s, respectively. This indicates that in tool-augmented reinforcement learning, the tool-call budget affects not only the policy space and interaction complexity, but also directly amplifies the computational and system overhead of training through longer interaction trajectories, more frequent external executions, denser sandbox calls, and increased synchronization waiting time. In particular, under the fixed high-budget setting Static-C=4, the cost of a single training step increases sharply, implying that under the same training-step budget, the total training cost would be significantly higher. Moreover, longer trajectories often come with greater interaction noise and a higher risk of instability, which is consistent with the earlier observation that fixed high budgets are more likely to cause oscillation or even collapse in the middle and later stages of training.

Overall, Table 3 further supports the necessity of progressive tool-call scheduling from the perspective of training cost. Maintaining a relatively low budget in the early stage can substantially reduce per-step time and noise, thereby improving training stability. Once the policy has become sufficiently mature, gradually relaxing the budget makes it possible to unlock the performance gains of multi-step tool coordination while still preserving training stability and controllable cost.

5.3. Analysis of Reasoning Performance

This section further analyzes the impact of different tool-budget strategies on the model's reasoning capability from two perspectives. First, we examine how different budget strategies affect final reasoning performance. Second, we analyze the evolution of tool-interaction behavior and code-generation behavior during training, thereby revealing how the progressive budget strategy improves reasoning ability while maintaining stable training.

From the perspective of final accuracy, the progressive tool-call scheduling method shows clear advantages on challenging mathematical reasoning tasks. As shown in Fig. 6, the progressive budget achieves the best results on both AIME24 and AIME25. Specifically, it reaches 36.7% on AIME24, outperforming both Static-C=1 at 33.3% and Static-C=4 at 30.0%; on AIME25, it reaches 30.0%, again exceeding Static-C=1 at 26.7% and Static-C=4 at 20.0%. These results indicate that in tool-augmented reinforcement learning, the intuition that a larger tool-call budget is always better does not hold. If a high upper bound on tool calls is introduced too early in training, the model tends to incur stronger exploration noise, which in turn interferes with the stability of policy learning.

To further analyze the source of the performance differences discussed above, this paper examines the performance dynamics during training. Fig. 7 shows the evolution of accuracy with training steps under different budget strategies on the AIME24 benchmark. It can be observed that the three

strategies exhibit clear differences in both training stability and performance evolution.

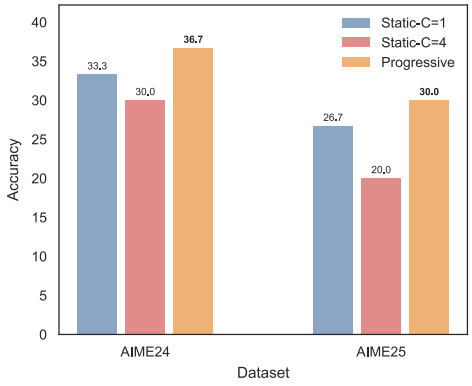


Fig. 6. Final accuracy under different tool-budget strategies.

Specifically, Static-C=4 becomes obviously unstable after around step 300 and terminates early, indicating that directly introducing a high tool-call budget in the early stage of training can substantially amplify the noise induced by long reasoning trajectories and thus push policy updates into an unstable regime. By contrast, although Static-C=1 can be trained stably up to 1000 steps, its performance improvement gradually slows down in the middle and later stages, and some fluctuation and decline emerge in the later phase of training. This suggests that when the budget constraint is too strict, the model may gradually develop a demand for multi-step tool interaction but remain constrained by the single-call limit, which in turn hinders further improvement in reasoning capability. In comparison, the progressive tool-call scheduling method exhibits a more stable performance-evolution trajectory throughout training and reaches a higher performance peak in the later stage. Its accuracy curve continues to improve in the middle and later phases and enters a higher range, which is consistent with the final performance trend shown in Fig. 6.

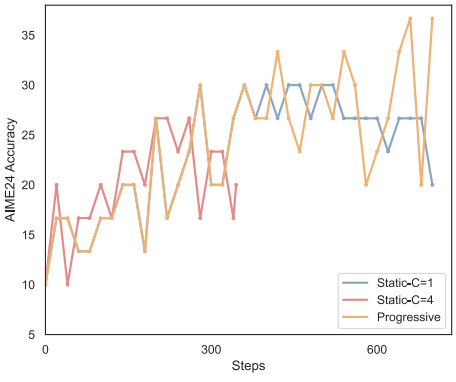


Fig. 7. Accuracy dynamics on AIME24 during training.

After clarifying the differences in performance dynamics, we further analyze the behavioral differences among strategies from the perspective of tool interaction. Fig. 8 shows the evolution of average tool-call intensity during training on the AIME24 and AIME25 datasets. Overall, all three budget strategies exhibit a rapid upward trend in the early stage of training, indicating that the model can quickly learn the value of tool invocation in mathematical reasoning. However, clear divergences gradually emerge in the middle and later stages of training.

For Static-C=1, the tool-call intensity continues to rise in the middle and later stages, suggesting that even when the budget is restricted to a single call, the model still exhibits a growing demand for multi-round tool interaction at the generation-policy level. This is consistent with the previously observed phenomenon of "tool-call intention exceeding the budget." By contrast, Static-C=4 shows a noticeably higher tool-call intensity in the early stage, but its curve terminates around step 300 due to instability in training. In comparison, the progressive tool-call scheduling method maintains a relatively smooth upward trend over a longer training interval and enters a higher-intensity regime after the budget is gradually relaxed. This indicates that the newly introduced budget can be progressively absorbed by the model and transformed into stable multi-round tool-interaction behavior. This evolutionary pattern further explains why the progressive method can achieve higher reasoning performance while preserving training stability.

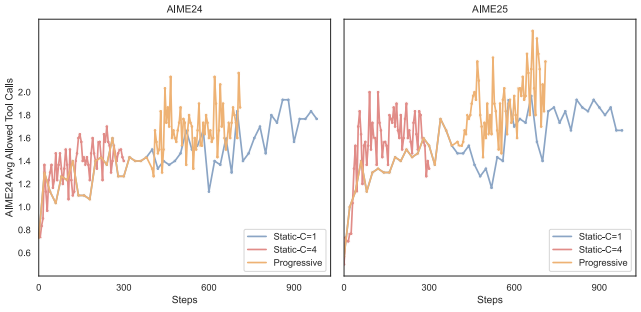


Fig. 8. Evolution of average tool-call intensity during training.

Beyond the number of tool calls, tool-use behavior also directly affects the structure of the model's outputs. Therefore, we further analyze changes in output form through the code-generation ratio, as shown in Fig. 9. Overall, under all three strategies, the code-generation ratio gradually increases as training proceeds, indicating that the model progressively learns to convert part of the reasoning process into executable code. However, the three budget strategies exhibit markedly different evolutionary patterns.

Static-C=4 shows a faster increase in the code-generation ratio in the early stage of training, suggesting that a higher budget more readily induces the model to generate longer code trajectories. At the same time, this also makes the model more prone to entering a high-noise interaction regime, thereby destabilizing training. By contrast, Static-C=1 exhibits a clear rise in the code-generation ratio in the middle and later stages, indicating that under a restricted tool-call budget, the model may compensate for insufficient interaction depth by generating longer code segments, which to some extent leads to higher generation cost. In comparison, the progressive budget strategy maintains a relatively moderate code-generation intensity in the early stage of training, thereby preventing the model from entering a high-noise long-trajectory interaction regime too early. In the

later stage, once the budget is gradually relaxed, the code-generation ratio rises steadily and remains in a relatively high range. This evolutionary pattern, namely first learning stable tool use and then gradually strengthening multi-step interaction capability, leads to a more continuous and controllable development of code-generation behavior. From the behavioral perspective, this further explains the performance gains brought by the progressive budget strategy.

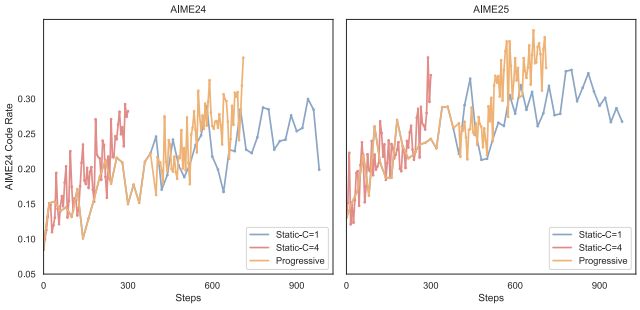


Fig. 9. Variation in the code-generation ratio with training steps.

Overall, a joint analysis of training-performance dynamics, tool-interaction behavior, and code-generation behavior shows that the progressive tool-call scheduling method not only achieves better final accuracy, but also induces a more stable trajectory of policy evolution throughout training. Compared with fixed-budget strategies, the progressive scheduling method controls interaction complexity in the early stage of training and gradually releases tool-use capacity in the later stage. This enables the model to explore more complex multi-step tool-interaction behavior while stably learning basic strategies, thereby achieving a more efficient and more stable improvement in tool-augmented reasoning capability.

5.4. Case Study

In the previous subsection, this paper analyzed, at the level of overall experimental results, the differences among tool-budget strategies in training dynamics, tool-interaction behavior, and code-generation behavior. To further understand the mechanism by which tool-augmented reasoning functions in concrete problem-solving processes, this subsection provides a qualitative analysis of model behavior through representative reasoning cases. Compared with the statistical analysis of macro-level indicators, case studies can reveal, from the perspective of a single reasoning trajectory, how the model uses tool-execution results to identify errors and revise its reasoning during problem solving, thereby forming a closed-loop reasoning pattern of generation, execution, feedback, and correction.

5.4.1. Tool-Based Error Correction

Fig. 10 presents a representative case of tool-based error correction. In this problem, the model first constructs a solution strategy through natural language reasoning and then uses the SymPy library to formulate equations and solve for the variables. However, when reading the returned tool result, the model mistakenly treats the returned object as a dictionary and attempts to access the solution through variable-name indexing, which triggers a `TypeError`. After receiving the execution feedback, the model is able to infer from the error message that the returned data structure is inconsistent with its expectation. It then revises the code accordingly by changing the result-reading method from variable-name indexing to positional indexing over the solution vector. The corrected code is subsequently re-executed, and the model successfully obtains the correct result, ultimately deriving the answer $x + y = 36$.

This case shows that within the tool-augmented reasoning framework, tool-execution results are not only used to complete complex computations, but also serve as verifiable feedback signals that help the model identify errors in intermediate reasoning or program implementation. By leveraging execution feedback, the model can make targeted revisions to local reasoning steps and thereby gradually approach the correct solution. This localized error-correction mechanism based on tool feedback enables the model to develop stable self-correction capability when solving complex computational problems.

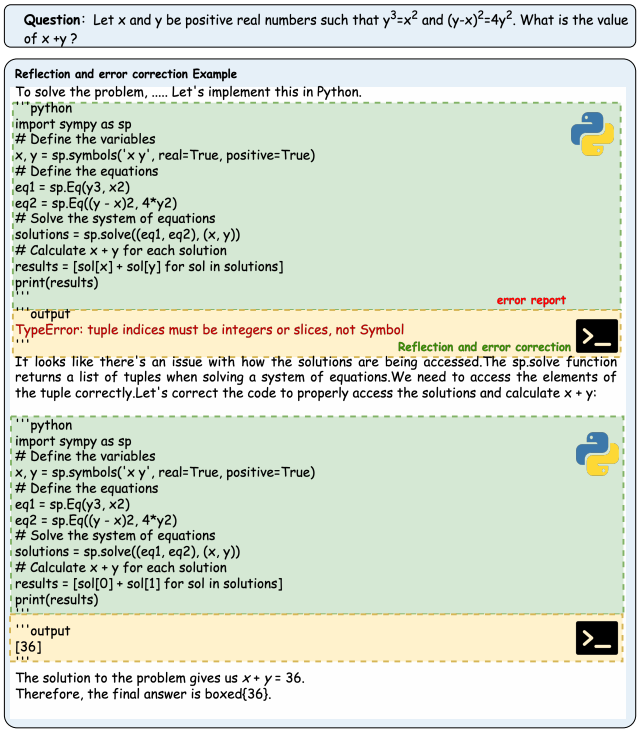


Fig. 10. Error correction driven by tool-execution feedback.

5.4.2. Logical Error Correction

In addition to program-execution errors, tool invocation can also help the model detect and correct logical errors in the reasoning process. Fig. 11 shows a representative case of logical error correction. In this problem, the model first relies on natural language reasoning to derive a preliminary answer. However, in order to verify the correctness of the reasoning process, the model subsequently generates code to check the derived result through computation. During tool execution, the computed result is found to be inconsistent with the answer obtained in the earlier reasoning process. This feedback prompts the model to re-examine its preceding reasoning steps. By further analyzing the problem

constraints and combining them with the code-execution result, the model identifies the logical error in its original reasoning and correspondingly adjusts its reasoning path. The model then regenerates a solution that satisfies the problem constraints and verifies it through the tool, ultimately obtaining the correct answer.

This case shows that tool execution not only compensates for the limitations of language models in complex computation, but also serves as a verifier during reasoning. When the reasoning result generated by the model is inconsistent with the result computed by the tool, execution feedback can trigger the model to re-evaluate its reasoning steps and thereby achieve self-correction at the logical level.

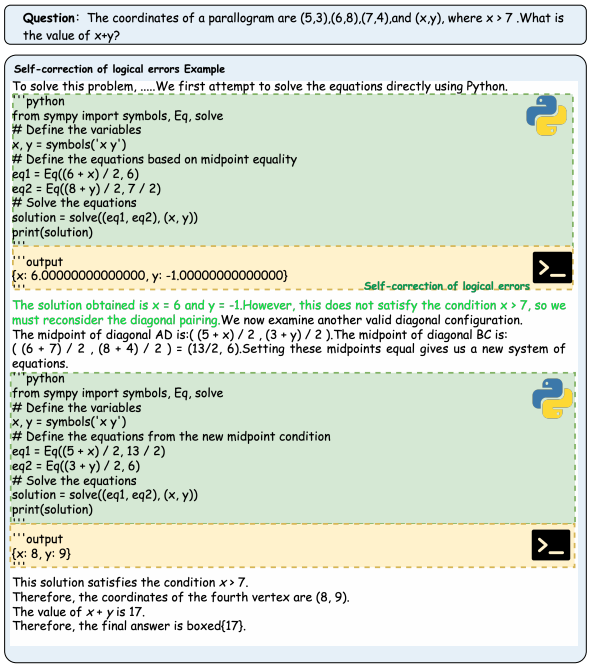


Fig. 11. Self-correction of logical errors.

The above cases show that in tool-augmented reasoning, external tools not only serve a computational function, but also provide important mechanisms for verification and feedback within the reasoning process. On the one hand, tool execution helps the model identify and correct errors at the level of program implementation. On the other hand, by externally validating reasoning results, the model can detect and correct deviations in logical derivation. Through this interaction pattern of reasoning, execution, feedback, and correction, the model is able to gradually improve the stability and reliability of solving complex problems. This further explains the performance gains brought by the progressive tool-call scheduling strategy observed in the preceding experiments.

6. Conclusion

To address the early policy oscillation and training instability that LLMs often exhibit during multi-round interaction in tool-augmented reinforcement learning, this paper proposes a progressive tool-call scheduling mechanism. By coupling the upper bound on tool calls with the global training process, the proposed method enables a stepwise dynamic relaxation of the number of allowed tool calls. In the early stage of training, the number of interactions is strictly constrained to reduce environmental noise; in the middle and later stages, the constraint is gradually relaxed to guide the model to learn multi-step verification and self-correction strategies based on external code-execution feedback. Experimental results show that on the Qwen2.5-Math-1.5B base model [15], the proposed method achieves an average accuracy of 56.33%, improving by 4.78% over the best fixed-budget baseline, with especially notable gains on challenging tasks such as AIME. At the same time, compared with static high-budget settings, the proposed method yields a smoother and more stable training process.

Although the progressive scheduling strategy proposed in this paper has achieved promising results on mathematical reasoning tasks, there remains substantial room for further extension. Future work may proceed in three directions. First, adaptive scheduling mechanisms based on task complexity or intrinsic rewards may be explored to enable on-demand allocation of tool-call budgets. Second, the method may be extended to larger-scale models in order to further investigate the relationship between model scale and high-frequency tool interaction. Third, the integration of tool-execution feedback with fine-grained process reward models may be studied to alleviate the long-sequence optimization difficulties caused by sparse rewards.

Acknowledgments

This work was supported by Major Science and Technology Project of Yunnan Province [No.202402AD080002]; and Yunnan Province Science and Technology Plan-Construction Project of Scientific and Technological Innovation Base [No.202607AB040002]; and Yunnan Province Expert Workstation Project [No.202605AF350050].

References

[1] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

[2] Changle Qu, Sunhao Dai, Xiaochi Wei, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Jun Xu, and Ji-Rong Wen. Tool learning with large language models: A survey. *Frontiers of Computer Science*, 19(8): 198343, 2025.

[3] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. Pal: Program-aided language models. In *International conference on machine learning*, pages 10764–10799. PMLR, 2023.

[4] Wenhu Chen, Xueguang Ma, Xinyi Wang, and William W Cohen. Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks. *arXiv preprint arXiv:2211.12588*, 2022.

[5] Zhibin Gou, Zhihong Shao, Yeyun Gong, Yelong Shen, Yujiu Yang, Minlie Huang, Nan Duan, and Weizhu Chen. Tora: A tool-integrated reasoning agent for mathematical problem solving. *arXiv preprint arXiv:2309.17452*, 2023.

[6] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda,

and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in neural information processing systems*, 36:68539–68551, 2023.

[7] Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, et al. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*, 2023.

[8] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948*, 2025.

[9] OpenAI. Learning to reason with llms. https://openai.com/research/learning-to-reason-with-llms, 2023. OpenAI Research blog, accessed April 2, 2026.

[10] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

[11] Xuefeng Li, Haoyang Zou, and Pengfei Liu. Torl: Scaling tool-integrated rl. *arXiv preprint arXiv:2503.23383*, 2025.

[12] Jiazhan Feng, Shijue Huang, Xingwei Qu, Ge Zhang, Yujia Qin, Baoquan Zhong, Chengquan Jiang, Jinxin Chi, and Wanjun Zhong. Retool: Reinforcement learning for strategic tool use in llms. *arXiv preprint arXiv:2504.11536*, 2025.

[13] Xinji Mai, Haotian Xu, Zhong-Zhi Li, Weinong Wang, Jian Hu, Yingying Zhang, Wenqiang Zhang, et al. Agent rl scaling law: Agent rl with spontaneous code execution for mathematical problem solving. *arXiv preprint arXiv:2505.07773*, 2025.

[14] Fei Bai, Yingqian Min, Beichen Zhang, Zhipeng Chen, Xin Zhao, Lei Fang, Zheng Liu, Zhongyuan Wang, and Hongteng Xu. Towards effective code-integrated reasoning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 30022–30030, 2026.

[15] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, et al. Qwen2. 5-math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024.

[16] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

[17] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Tom Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in neural information processing systems*, 36:11809–11822, 2023.

[18] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*, 2022.

[19] Peiyi Wang, Lei Li, Zhihong Shao, Runxin Xu, Damai Dai, Yifei Li, Deli Chen, Yu Wu, and Zhifang Sui. Math-shepherd: Verify and reinforce llms step-by-step without human annotations. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9426–9439, 2024.

[20] Ganqu Cui, Lifan Yuan, Zefan Wang, Hanbin Wang, Yuchen Zhang, Jiacheng Chen, Wendi Li, Bingxiang He, Yuchen Fan, Tianyu Yu, et al. Process reinforcement through implicit rewards. *arXiv preprint arXiv:2502.01456*, 2025.

[21] Ke Wang, Houxing Ren, Aojun Zhou, Zimu Lu, Sichun Luo, Weikang Shi, Renrui Zhang, Linqi Song, Mingjie Zhan, and Hongsheng Li. Mathcoder: Seamless code integration in llms for enhanced mathematical reasoning. *arXiv preprint arXiv:2310.03731*, 2023.

[22] Debrup Das, Debopriyo Banerjee, Somak Aditya, and Ashish Kulkarni. Mathsensei: a tool-augmented large language model for mathematical reasoning. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 942–966, 2024.

[23] Nuo Chen, Hongguang Li, Baoyuan Wang, and Jia Li. From good to great: Improving math reasoning with tool-augmented interleaf prompting. In *Proceedings of the 2nd Workshop on Natural Language Reasoning and Structured Explanations (@ ACL 2024)*, pages 64–79, 2024.

[24] Chengpeng Li, Mingfeng Xue, Zhenru Zhang, Jiaxi Yang, Beichen Zhang, Bowen Yu, Binyuan Hui, Junyang Lin, Xiang Wang, and Dayiheng Liu. Start: Self-taught reasoner with tools. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 13523–13564, 2025.

[25] Joykirat Singh, Raghav Magazine, Yash Pandya, and Akshay Nambi. Agentic reasoning and tool integration for llms via reinforcement learning. *arXiv preprint arXiv:2505.01441*, 2025.

[26] Wenxun Wu, Yuanyang Li, Guhan Chen, Linyue Wang, and Hongyang Chen. Tool-augmented policy optimization: Synergizing reasoning and adaptive tool use with reinforcement learning. *arXiv preprint arXiv:2510.07038*, 2025.

[27] Chujie Zheng, Shixuan Liu, Mingze Li, Xiong-Hui Chen, Bowen Yu, Chang Gao, Kai Dang, Yuqiong Liu, Rui Men, An Yang, et al. Group sequence policy optimization. *arXiv preprint arXiv:2507.18071*, 2025.

[28] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

[29] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[30] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset. *arXiv preprint arXiv:2103.03874*, 2021.

[31] Weihao Zeng, Yuzhen Huang, Qian Liu, Wei Liu, Keqing He, Zejun Ma, and Junxian He. Simplerl-zoo: Investigating and taming zero reinforcement learning for open base models in the wild. *arXiv preprint arXiv:2503.18892*, 2025.